

Solution TD4

Exercice 01:

```
#define M 10          // Taille du tampon
semaphore S = 0,      // Un sémaphore pour contrôler l'accès à l'impression
mutex = 1 ;          // Un sémaphore d'exclusion mutuelle pour l'accès au tampon
int nb_msg = 0;       // Nombre de messages déposés*/
```

<pre>void Pi(){ /* i={1..N} <Générer un message> wait(mutex) <Déposer le message> nb_msg = nb_msg + 1 if (nb_msg == M) signal(S) else signal(mutex) }</pre>	<pre>void Ps(){ while(1){ wait(S) <Imprimer tous les messages> nb_msg = 0 signal(mutex) } }</pre>
---	---

Exercice 02: Solution du problème de coiffeur endormi (en utilisant les sémaphores)

```
#define N 10          // nombre de chaises dans le salon
int attente ;        // nombre de clients en attente
semaphore mutex =1 ; // exclusion mutuelle sur la variable 'attente'
semaphore coiffeur =0 ;
semaphore client =0 ;
```

```
void coiffeur()
{
    while (1) {

        wait (client) ;           // dormir si pas de client
        wait (mutex);
        attente = attente -1;
        signal (coiffeur);
        signal (mutex);           // le coiffeur est disponible
        Couper_Cheveux ();

    }
}
```

```

void client ()
{

wait (mutex);
if (attente < N) {

    attente = attente +1;
    signal (client);    //réveiller le coiffeur
    signal (mutex);
    wait (coiffeur) ;
    Se_Faire_couper_Les_cheveux() ;
    }
else signal (mutex);

}

int main( ){
    attente =0 ;
    parbegin (coiffeur(), client(), client(),.....,client() ) ;
    return 0 ;

}

```

Exercice 03 :

semaphore s_calc = 1 /* mutex protégeant l'accès à la calculatrice */
semaphore s_stylo = 1 /* mutex protégeant l'accès au stylo */
semaphore s_cours=1 /* mutex protégeant l'accès au support de cours*/

Candidat 1	Candidat 2
<pre> void Candidat_1(){ wait(s_calc); utiliser_calc(); signal(s_calc); wait (s_stylo); wait (s_cours); utiliser_stylo_cours(); signal (s_cours); signal (s_stylo); } </pre>	<pre> void Candidat_2(){ wait (s_calc); wait (s_stylo); utiliser_calc_stylo(); signal (s_stylo); signal (s_calc); wait (s_cours); utiliser_cours(); signal (s_cours); } </pre>

Exercice 4 :

```
semaphore mutex =1 ;  
semaphore enseignant =0,  
        agent =0 ;  
int enAttente = 0 ;
```

```
void Agent() {
```

```
    while(1){  
        wait(enseignant)  
        wait (mutex)  
        enAttente --  
        signal(agent)  
        signal(mutex)
```

```
    tirer_examen();
```

```
    }  
}
```

```
void enseignant() {
```

```
    wait (mutex)
```

```
    if(enAttente<4) {
```

```
        enAttente++  
        signal (enseignant)  
        signal (mutex)  
        wait (agent)
```

```
        participer_tirage_examen() ;
```

```
    }
```

```
    else
```

```
        signal(mutex)
```

```
    }
```